

Rapport de Soutenance 1 : HarmonyStudio

Projet S4 – EPITA

Alexandre d'AVEZAC de CASTERA

Léa-Angéline KOLMERSCHLAG

Hugo GUYENNET

Quentin JOLY

Table des matières

1	Rôles et missions des membres	2
1.1	Alexandre - Moteur DSP & synthèse sonore	2
1.2	Quentin - Séquençage & structures de données temporelles	2
1.3	Léa - Interface graphique	2
1.4	Hugo - Système de fichiers & export audio	2
2	Objectifs, planification et état d'avancement	3
2.1	Objectifs de la Soutenance 1	3
2.2	Planning prévu	3
2.3	État d'avancement réel	3
3	Difficultés, choix technologiques et algorithmiques	4
3.1	Moteur audio temps réel	4
3.1.1	Choix technologiques	4
3.1.2	Choix algorithmiques	4
3.1.3	Difficultés rencontrées	4
3.2	Synthèse sonore	6
3.2.1	Chaîne de signal	6
3.2.2	Génération de formes d'ondes : PolyBLEP	6
3.2.3	Enveloppes ADSR	6
3.2.4	Filtrage numérique : filtre biquad	7
3.2.5	Polyphonie et voice stealing	7
3.2.6	Effets : delay et réverbération de Schroeder	7
3.2.7	Difficultés rencontrées	7
3.3	Séquençage temporel	9
3.3.1	Choix technologiques	9
3.3.2	Choix algorithmiques	9
3.3.3	Difficultés rencontrées	9
3.4	Interface graphique	11
3.4.1	Choix technologiques	11
3.4.2	Difficultés rencontrées	11
3.5	Gestion des fichiers et export audio	13
3.5.1	Choix technologiques	13
3.5.2	Difficultés rencontrées	13
4	Bibliographie	14

1 Rôles et missions des membres

1.1 Alexandre - Moteur DSP & synthèse sonore

Alexandre se concentre sur le moteur DSP, domaine exigeant une maîtrise approfondie du traitement du signal et des mathématiques associées. Son travail englobe l'implémentation des oscillateurs, des enveloppes ADSR, des filtres numériques, la gestion de la polyphonie ainsi que les effets audio (delay et réverbération). Il est également responsable de l'architecture du thread audio temps réel et de la gestion des buffers circulaires.

1.2 Quentin - Séquençage & structures de données temporelles

Quentin prend en charge la logique du séquenceur et les structures de données temporelles. Cette responsabilité inclut la représentation interne des événements musicaux, les algorithmes de planification des steps, la synchronisation avec le moteur audio ainsi que la gestion des notes de type MIDI. La coordination entre le séquenceur et le moteur audio constitue un point critique nécessitant une collaboration étroite avec Alexandre.

1.3 Léa - Interface graphique

Léa développe l'interface graphique, tâche complexe impliquant la création d'éditeurs visuels pour la timeline, les paramètres d'instruments et les contrôles de transport. Son travail repose sur la bibliothèque `egui` (Immediate Mode GUI). L'ergonomie et la réactivité de l'interface détermineront largement l'expérience utilisateur finale.

1.4 Hugo - Système de fichiers & export audio

Hugo assure la gestion du système de fichiers, l'import et l'export au format JSON pour la persistance des projets, ainsi que l'exportation audio au format WAV via la bibliothèque `hound`. Cette couche inclut également la gestion des erreurs d'entrée/sortie et la validation des formats de fichiers pour garantir la robustesse de l'application.

2 Objectifs, planification et état d'avancement

2.1 Objectifs de la Soutenance 1

La première soutenance vise à démontrer la viabilité technique du projet avec un prototype fonctionnel capable de générer et de lire du son . Les objectifs fixés dans le cahier des charges sont les suivants :

- Moteur audio opérationnel (thread dédié, communication avec `cpal`, buffers circulaires stables) ;
- Synthèse de base fonctionnelle (oscillateurs simples, enveloppes ADSR) ;
- Séquenceur permettant le déclenchement programmatique de notes ;
- Interface graphique minimale (play, visualisation du piano) ;
- Export MP3 fonctionnel via `mp3lame-encoder`.

2.2 Planning prévu

Tâche	Début prévu	Fin prévue	Responsable
Moteur audio	17/01/2026	07/03/2026	Alexandre
Oscillateurs de base	17/01/2026	07/03/2026	Alexandre
Enveloppes ADSR	17/01/2026	14/03/2026	Alexandre
Structures de données séquenceur	17/01/2026	11/03/2026	Quentin
UI minimale (egui)	17/01/2026	13/03/2026	Léa
Export WAV	17/01/2026	14/03/2026	Hugo
Site web (version 1)	17/01/2026	23/03/2026	Quentin

2.3 État d'avancement réel

Tâches	Objectif S1	Réel S1
Moteur Audio (DSP)	100%	100%
Synthèse d'instruments	20%	30%
Séquençage temporel	50%	75%
Interface graphique	40%	60%
Système de fichiers	10%	20%
Site Web	40%	70%

TABLE 1 – Avancement par composant à la Soutenance 1

3 Difficultés, choix technologiques et algorithmiques

3.1 Moteur audio temps réel

Le moteur audio constitue le noyau du système. Il s’articule autour de trois couches distinctes : la couche matérielle gérée par `cpal`, le séquenceur (`StepSequencer`) qui orchestre les pistes, et les moteurs d’instruments (`InstrumentEngine`) qui produisent les échantillons.

3.1.1 Choix technologiques

Le point d’entrée public est `play.rs`, qui expose des fonctions simples (`play`, `play_chord`, `play_melody`) au reste de l’application. Toute la configuration de `cpal`, la gestion des threads et le câblage du séquenceur sont encapsulés dans ce fichier et ne sont jamais exposés directement.

La bibliothèque `cpal` est configurée avec une fréquence d’échantillonnage de 44 100 Hz, un buffer de 1 024 échantillons et une sortie stéréo deux canaux. Une fois le stream lancé, `cpal` appelle une closure environ 44 100 fois par seconde depuis un thread système dédié. Cette callback est le seul endroit où du son est produit : pour chaque frame stéréo, elle appelle `StepSequencer::next_sample()` et copie le résultat sur les deux canaux.

3.1.2 Choix algorithmiques

Il est impératif que cette callback ne bloque jamais et n’alloue pas de mémoire dynamique, sous peine de provoquer des coupures audio. Toutes les structures sont donc pré-allouées à l’initialisation.

À chaque appel de `next_sample()`, le séquenceur avance d’un échantillon : il met à jour `sample_clock`, déclenche les `note_on` et `note_off` aux bons instants, puis délègue la génération du son à chaque `InstrumentEngine`. Chaque piste possède son propre engine, qui agrège un `VoicePool` de 32 voix et une `EffectsChain`. Le pipeline par piste est fixe :

- sommation de toutes les voix actives du pool (avec gain maître) ;
- application du delay (buffer circulaire + feedback) ;
- application de la réverbération (réseau de Schroeder).

Les échantillons de toutes les pistes sont ensuite sommés pour produire l’échantillon final.

La communication entre le thread audio et le thread principal repose sur un `Arc<Mutex< StepSequencer> >`. Le thread audio verrouille le mutex à chaque callback pour lire les commandes et produire les échantillons. Cette approche est fonctionnelle dans le cadre du projet, bien qu’un vrai DAW utiliserait des ring buffers SPSC lock-free pour garantir l’absence totale de blocage du thread audio.

3.1.3 Difficultés rencontrées

Plusieurs difficultés ont été rencontrées lors de la mise en place du moteur audio. La principale concerne le modèle de programmation imposé par `cpal`, en particulier le paradigme callback-based qui contraint fortement la structure du code. La callback ne pouvant ni bloquer ni allouer de mémoire, plusieurs refactorisations ont été nécessaires pour déplacer toutes les initialisations en dehors du chemin critique.

Des problèmes de synchronisation entre le thread audio et le thread principal ont également été rencontrés, se manifestant par des coupures sonores ou des comportements non déterministes lors de changements de paramètres en cours de lecture. La gestion correcte du **Mutex** et l'ordre des opérations ont demandé plusieurs itérations avant d'obtenir un comportement stable.

3.2 Synthèse sonore

3.2.1 Chaîne de signal

Chaque voix active suit une chaîne de traitement fixe : **Oscillateur** → **Filtre biquad** → **Enveloppe ADSR** → sortie. Cette ordre n'est pas anodin : filtrer avant d'appliquer l'enveloppe permet de sculpter le timbre indépendamment du volume, ce qui correspond au comportement des synthétiseurs analogiques classiques. La sortie de chaque voix passe ensuite par un **tanh** pour limiter les saturations sans écrêtage brutal, puis toutes les voix actives sont sommées et multipliées par un gain maître inversement proportionnel au nombre de voix ($\text{MASTER_GAIN} = 1 / (\text{MAX_VOICES} / 4)$), ce qui évite les débordements de dynamique lors de l'utilisation intensive de la polyphonie.

3.2.2 Génération de formes d'ondes : PolyBLEP

La génération naïve de formes d'ondes non-sinusoïdales introduit systématiquement de l'aliasing : les discontinuités abruptes d'une onde carrée ou en dents de scie produisent des composantes fréquentielles au-delà de la fréquence de Nyquist, qui se replient dans le spectre audible sous forme d'artefacts métalliques.

Pour y remédier, nous implémentons la technique **PolyBLEP** (Polynomial Band-Limited Step). Lors de la traversée d'une discontinuité, un polynôme de correction est calculé localement et ajouté à la forme d'onde naïve, lissant la transition sans affecter le reste du cycle. Le calcul est minimal : seulement une à deux vérifications de phase par échantillon, ce qui le rend particulièrement adapté au traitement temps réel.

Par ailleurs, l'oscillateur repose sur une architecture **triple unisson** : trois oscillateurs légèrement désaccordés (détune de $\pm 0.2\%$) sont moyennés, produisant un son naturellement plus riche sans coût algorithmique significatif. Quatre formes d'ondes sont disponibles : sinusoïdale, en dents de scie, carrée et triangulaire. L'onde triangulaire est dérivée par intégration d'une onde carrée (leaky integrator), ce qui lui confère une nature intrinsèquement band-limitée.

3.2.3 Enveloppes ADSR

L'enveloppe ADSR modélise l'évolution temporelle de l'amplitude d'une note selon quatre phases : attaque, décroissance, sustain et relâchement. Plutôt que des rampes linéaires, nous utilisons des **courbes exponentielles** basées sur un filtre passe-bas du premier ordre (one-pole filter) :

$$\text{level}[n] = \text{target} + (\text{level}[n - 1] - \text{target}) \times \text{coeff}$$

avec $\text{coeff} = e^{-1/(t_s \times f_s)}$. Ce comportement correspond mieux à la perception humaine de l'intensité sonore, qui est elle-même logarithmique.

Chaque voix possède sa propre instance d'enveloppe, indépendante des autres, ce qui permet une polyphonie complète avec des notes à des stades d'évolution différents. Une machine à états explicite (**Idle**, **Attack**, **Decay**, **Sustain**, **Release**) gère les transitions et signale quand la voix peut être libérée.

3.2.4 Filtrage numérique : filtre biquad

Les filtres biquad (bi-quadratiques) implémentent une équation aux différences du second ordre :

$$y[n] = b_0x[n] + b_1x[n-1] + b_2x[n-2] - a_1y[n-1] - a_2y[n-2]$$

Ils offrent un excellent compromis entre coût computationnel (5 multiplications et 4 additions par échantillon) et polyvalence : passe-bas, passe-haut, passe-bande et notch sont tous obtenus en modifiant les cinq coefficients $\{b_0, b_1, b_2, a_1, a_2\}$, calculés selon les formules de l'*Audio EQ Cookbook* de Robert Bristow-Johnson via la transformation bilinéaire.

Lorsque les paramètres changent (fréquence de coupure, facteur de qualité Q), les coefficients sont interpolés linéairement sur 64 échantillons (1.5 ms à 44.1 kHz) pour éviter tout clic audible. Les lignes de délai internes sont réinitialisées à chaque `note_on` pour éviter le bruit résiduel d'une note précédente.

3.2.5 Polyphonie et voice stealing

La polyphonie repose sur un pool de 32 voix pré-allouées (`VoicePool`). Lors du déclenchement d'une note, le système cherche d'abord une voix libre. Si toutes sont actives, une stratégie de **voice stealing** est appliquée : les voix en phase de Release sont prioritairement interrompues, car elles s'éteignent naturellement et leur interruption est peu perceptible. En l'absence de telles voix, la voix la plus ancienne (champ `age` incrémenté à chaque échantillon) est sélectionnée.

3.2.6 Effets : delay et réverbération de Schroeder

Le moteur intègre deux effets temporels organisés en chaîne séquentielle (`EffectsChain`) : le signal passe d'abord dans le delay puis dans la réverbération.

Le **delay** utilise un buffer circulaire de deux secondes maximum. Le signal retardé est réinjecté en entrée avec un coefficient de feedback, créant des échos répétés qui décroissent naturellement. La taille du buffer détermine le temps de retard, et le mix sec/mouillé est contrôlable indépendamment.

La **réverbération** implémente le réseau de Schroeder (1962) : quatre filtres en peigne parallèles simulent les premières réflexions d'un espace acoustique, puis deux filtres passe-tout en série diffusent le signal pour créer une queue de réverbération dense. Les délais des filtres en peigne sont choisis premiers entre eux pour éviter les résonances périodiques. Bien que plus simple que des algorithmes modernes comme Freeverb, cette approche produit une réverbération musicalement utilisable avec un coût computationnel très faible.

3.2.7 Difficultés rencontrées

La principale difficulté a été de comprendre le fonctionnement du son numérique pour pouvoir le reproduire, ainsi que d'assimiler les articles et ressources d'autres auteurs sur le sujet. Une fois ces bases acquises, l'application et l'adaptation des algorithmes à nos besoins a pu commencer, bien que de nombreuses impasses algorithmiques aient été rencontrées en cours de route. Le son produit se révélait par moments brouillon, trouble ou strident, nécessitant de nombreux ajustements des paramètres et de la chaîne de traitement.

Un autre problème notable concernait un dépassement mémoire au niveau du buffer audio sous Windows, qui limitait la durée de lecture à une vingtaine de secondes. Ce problème a finalement été identifié et résolu.

3.3 Séquençage temporel

L'objectif de cette partie est de concevoir et implémenter un step sequencer, c'est-à-dire un système permettant de programmer et déclencher des événements musicaux de manière discrète dans le temps, selon une grille régulière. Le séquenceur gère plusieurs pistes, permet l'activation et la désactivation de steps, déclenche des notes avec une hauteur et une vélocité, et fonctionne en temps réel dans la callback audio.

3.3.1 Choix technologiques

L'intégration avec le moteur audio repose sur un modèle **sample-based**, où la fonction principale du séquenceur est appelée à chaque échantillon audio. Ce choix permet une synchronisation précise entre la logique musicale et la génération du signal sonore.

Les structures principales sont les suivantes :

- **Step** : représente une case de la grille ;
- **Track** : représente une piste associée à un instrument ;
- **StepSequencer** : coordonne l'ensemble du système.

3.3.2 Choix algorithmiques

Le temps est géré en nombre d'échantillons audio plutôt qu'en unités musicales abstraites. Chaque step a une durée calculée en échantillons : `samples_per_step` dépend du BPM et du sample rate, tandis que `sample_clock` indique la position dans le step courant. Cela permet une précision temporelle maximale.

Au début de chaque step (`sample_clock == 0`), le séquenceur parcourt toutes les pistes, vérifie si le step courant est actif, et déclenche un `note_on` si nécessaire. La note jouée est soit celle définie dans le step, soit la note par défaut de la piste. La vélocité suit la même logique.

La durée des notes est gérée via un paramètre global `NOTE_GATE`. Les notes sont arrêtées automatiquement après un pourcentage du step : `note_off` est déclenché à `NOTE_GATE * samples_per_step`. Ce choix simplifie la gestion des notes et garantit un comportement rythmique stable.

À chaque échantillon, `sample_clock` est incrémenté. Lorsqu'il atteint `samples_per_step`, on passe au step suivant. En fin de séquence, le séquenceur boucle ou arrête la lecture selon la configuration. Le séquenceur ne génère pas directement le son : il délègue cette tâche aux `InstrumentEngine`, dont les échantillons sont sommés pour produire la sortie finale.

3.3.3 Difficultés rencontrées

Le principal enjeu a été de garantir une synchronisation précise avec le moteur audio. Le passage à une gestion en échantillons plutôt qu'en temps abstrait a permis de résoudre ce problème. Il a également été nécessaire de stocker la note en cours (`active_note`) afin de pouvoir envoyer correctement les événements `note_off` et éviter les notes bloquées. Enfin, les fonctions manipulant les steps utilisent des accès sécurisés (`get_mut`) pour éviter les erreurs d'index hors limites.

L'implémentation actuelle présente plusieurs limitations : la durée des notes est globale et non configurable par step, chaque piste est monophonique, et le mixage audio reste une somme directe sans normalisation. Des améliorations sont prévues pour la soutenance

finale, notamment l'ajout d'une durée de note par step et la gestion polyphonique par piste.

3.4 Interface graphique

3.4.1 Choix technologiques

Nous avons choisi d'utiliser la bibliothèque **eframe** pour implémenter l'interface graphique car elle permet de visualiser la fenêtre graphique à chaque frame. De plus, elle est facile et rapide d'utilisation.

L'interface graphique comporte trois parties distinctes afin de la rendre plus intuitive et facile d'utilisation.

La première partie se trouve tout en haut. Elle comporte trois combo box : une pour tout ce qui est relatif aux fichiers, une seconde pour la sélection de l'instrument de musique, et la dernière pour la sélection de l'effet. En plus de ces trois combo box, deux boutons sont disponibles : un bouton play pour lancer la musique et un bouton stop pour l'arrêter. Un bouton supplémentaire permet de tester tous les sons du piano.

La seconde partie se trouve à gauche et représente un piano permettant à l'utilisateur d'appuyer sur les touches pour tester les sons de l'instrument. Ce piano est réparti sur quatre pages afin d'améliorer l'expérience utilisateur. En effet, afficher le piano en entier sur une seule page comprimerait les touches au point de les rendre inexploitable. Ce piano aide également l'utilisateur à se repérer lorsqu'il souhaite créer sa propre partition.

La troisième partie est une grille de cases cochables permettant à l'utilisateur de créer sa propre partition. Chaque case est contenue dans une matrice liée à chaque instrument, ce qui permet de jouer les notes avec plusieurs instruments simultanément.

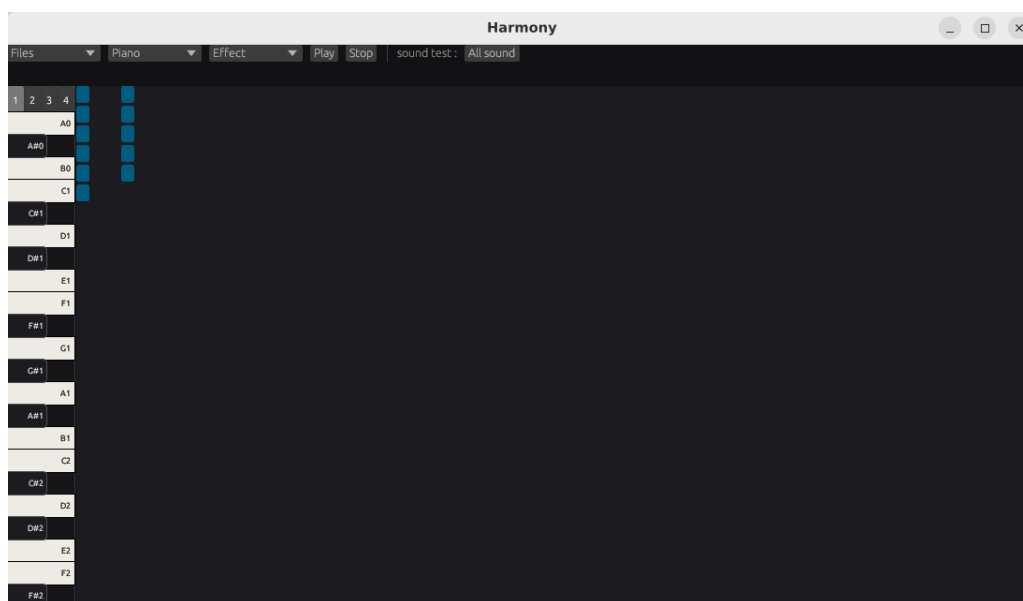


FIGURE 1 – Interface graphique d'HarmonyStudio à la Soutenance 1

3.4.2 Difficultés rencontrées

Des difficultés ont notamment été rencontrées lors de la création du piano. Le premier choix s'est porté sur une base de boutons cliquables, mais il s'est avéré qu'il n'était pas possible de les redimensionner. La solution retenue a donc été d'utiliser des rectangles cliquables et redimensionnables.

Comme on peut le voir sur l'image ci-dessus, les cases de la troisième partie ne sont pas encore totalement alignées avec le piano, et un espace important subsiste entre les cases. Ces points seront corrigés pour la soutenance finale.

3.5 Gestion des fichiers et export audio

Dans le cadre de HarmonyStudio, la gestion des fichiers constitue une partie essentielle du projet, car elle permet à l'utilisateur de sauvegarder, charger et surtout exporter sa création sous une forme exploitable. La partie exportation se compose actuellement de deux éléments principaux :

- l'importation et l'exportation au format JSON
- l'exportation audio au format WAV

3.5.1 Choix technologiques

L'import/export JSON repose sur la bibliothèque **Serde**, qui permet de sérialiser et désérialiser facilement les structures du projet. Cette conversion constitue une base pour la persistance des données, car elle garantit une bonne lisibilité des fichiers, notamment utile en phase de développement pour le débogage. De plus, le format JSON offre une grande flexibilité, permettant d'ajouter de nouveaux champs sans remettre en cause la compatibilité globale du projet.

Cependant, le format JSON n'est pas adapté au stockage de données volumineuses, en particulier pour l'audio brut. Il est donc utilisé uniquement pour représenter la structure du projet, et non les données audio elles-mêmes.

L'exportation en WAV représente la concrétisation du projet, puisqu'elle permet de générer un fichier audio final pouvant être écouté et partagé. Pour cela, chaque note active est convertie en une onde sinusoïdale. La fréquence du signal est calculée à partir de la note MIDI, tandis que la vitesse influence son amplitude. Les échantillons ainsi générés sont ensuite écrits dans un fichier audio avec une fréquence d'échantillonnage fixe. Cette partie s'appuie sur la bibliothèque **hound**, qui permet la création et l'écriture de fichiers WAV en Rust.

3.5.2 Difficultés rencontrées

Plusieurs difficultés ont été rencontrées. La principale concerne la génération du signal audio, notamment la conversion correcte entre note MIDI et fréquence, ainsi que la gestion temporelle des échantillons. De plus, l'implémentation actuelle reste séquentielle : les pistes ne sont pas encore mixées entre elles, ce qui constitue une limitation importante.

Une autre limite actuelle est l'absence de support du format MP3. Contrairement au WAV, ce format repose sur une compression avec perte, nécessitant l'intégration de bibliothèques plus complexes et une gestion plus avancée des données audio.

Enfin, certains choix ont été faits afin de privilégier une première version fonctionnelle du pipeline complet. L'export audio repose pour l'instant sur une synthèse simple, suffisante pour valider le fonctionnement global. Cette base pourra être améliorée par la suite, notamment avec l'ajout d'un véritable système de mixage et le support de nouveaux formats d'import/export comme le MP3.

4 Bibliographie

1. cpal – Cross-Platform Audio Library. <https://github.com/RustAudio/cpal>
2. Julius O. Smith III, *Introduction to Digital Filters with Audio Applications*, CCRMA, Stanford University, 2007. <https://ccrma.stanford.edu/~jos/filters/>
3. Nils Jonas Norberg, *PolyBLEP Oscillator*, 2010.
4. minimp3 – Rust bindings. <https://crates.io/crates/minimp3>
5. mp3lame-encoder – Rust bindings for LAME. <https://crates.io/crates/mp3lame-encoder>
6. egui – Immediate Mode GUI in Rust. <https://github.com/emilk/egui>
7. ringbuf – Lock-free SPSC ring buffer. <https://crates.io/crates/ringbuf>
8. Udo Zölzer (éd.), *Digital Audio Effects*, 2^e éd., Wiley, 2011.